INFORMATION RETRIEVAL PRACTICALS

INDEX

09	**Learning to Rank** • **Implement a learning to rank algorithm (eg. RanksVM or RankBoost).** • **Train the ranking model using labelled data and evaluate its effectiveness.**
10	**Advanced Topic in Information Retrieval** • **Implement a text summarization algorithm (eg. Extractive or abstractive).** • **Build a question-answering system using techniques such as information extraction.**

Practical-1

Document Indexing and Retrieval

- **Implement an inverted index construction algorithm.**
- **Build a simple document retrieval system using the constructed index.**

Program:-

```
import re

from collections import defaultdict

class DocumentRetrievalSystem:

def_init_(self):

 self.index=defaultdict(set)

self.documents={}


def tokenize(self, text):

return re.findall(r'\b\w+\b',text.lower())


def index_document(self, doc_id, text):

tokens=self.tokenize(text)

for token in tokens:

self.index[token].add(doc_id)

self.documents[doc_id]=text


def retrieve_documents(self, query):

tokens=self.tokenize(query)

relevant_doc_ids=set()

for token in tokens:

if token in self.index:
```

```
if not relevant_doc_ids:
relevant_doc_ids.update(self.index[token])
else:
relevant_doc_ids.intersection_update(self.index[token])
return [(doc_id, self.documents[doc_id])for doc_id in relevant_doc_ids]

doc_retrieval=DocumentRetrievalSystem()

doc_retrieval.index_document(1,"This is the first Document")
doc_retrieval.index_document(2,"This is the second Document")
doc_retrieval.index_document(3,"Another document is here")

query="first document"
results=doc_retrieval.retrieve_documents(query)

print("Query:", query)
for doc_id, text in results:
print(f"Document{doc_id}:{text}")
```

Output:-

Query:first document

Document 1:This is the first document

Practical-2

Retrieval Models

- **Implement the Boolean retrieval model and process queries.**
- **Implement the vector space model with TF-IDF weighting and cosine similarity.**

Program:-

```
import pandas as pd

from sklearn.feature_extraction.text import CountVectorizer

print("Demonstration of Boolean Retrieval Model using Bitwise operations on Term Document Invidence Matrix of a corpus")

corpus={

'this is the first document.',

'this document is the second document.',

'And this is the third one.',

'Is this the first document?'

}

print("The corpus is:")

print(corpus)

vectorizer=CountVectorizer()

x=vectorizer.fit_transform(corpus)

df=pd.DataFrame(x.toarray(),columns=vectorizer.get_feature_names())

print("This generated DataFrame is:")

print(df)

print("Query processing on thee term document incidence matrix:")

print("1. Find all document ids for query this AND first")

alldata=df[(df['this']==1)&(df['first']==1)]
```

```
print("Document ids where both 'this' and 'first' terms are present are:",alldata.index.tolist())

print("2. Find all document ids for query this OR first")

ordata=df[(df['this']==1)|(df['first']==1)]

print("Document ids where either of 'this' and 'first' terms are present are:", ordata.index.tolist())

print("3. Find all document ids for query NOT and")

notdata=df[(df['and']!=1)]

print("Document ids where 'And ' term is not present are:",notdata.index.tolist())
```

Output:-

The generated data frame:

	and	document	first	is	one	second	the	third	this
0	0	2	0	1	0	1	1	0	1
1	0	1	1	1	0	0	1	0	1
2	1	0	0	1	1	0	1	1	1
3	0	1	1	1	0	0	1	0	1

Query processing on term document incidence matrix:

1. First all document ids for query this AND first

Document ids where both 'this' and 'first' terms are present are: [1,3]

2. Find all document ids for query this OR first

Document ids where either of 'this' and 'first' terms are present are: [0,1,2]

3. Find all document ids for query NOT and

Document ids where 'And ' term is not present are: [0,1,3]

Practical-3

Spelling Correction in IR Systems

- **Develop a spelling correction module using edit distance algorithms.**
- **Integrate the spelling correction module into an information retrieval system.**

Program:-(3a)

```
def levenshtein_distance(str1,str2):
len_str1=len(str1)+1
len_str2=len(str2)+1

matrix=[[0 for_in range(len_str2)]for_in range(len_str1)]

for i in range(len_str1):
matrix[i][0]=i
for j in range(len_str2):
matrix[0][j]=j
for I in range(1, len_str1):
for j in range(1, len_str2):
cost=0 if str1[i-1]==str2[j-1] else 1
matrix[i][j]==min(
matrix[i-1][j]+1,
matrix[i][j-1]+1,
matrix[i-1][j-1]+cost)
return matrix[len_str-1][len_str2-1]
def suggest_correction(word,word_list):
```

```
distances=[(w,levenshtein_distance(word,w))for w in word_list]
distance.sort(key=lambda x:x[1])
return distance[0][0]
if__name__=="__main__":
input_word="helo"
dictionary=["hello","world","python","spell","correct","algorithm"]

suggested_correction=suggest_correction(input_word,dictionary)
print(f"Suggested correction for '{input_word}':{suggested_correction}")
```

Output:-

Suggested correction for 'helo':hello

Program:-(3b)

```
def retrieve_information(query, dictionary):

query_words=query_split()


corrected_words=[suggest_correction(word, dictionary)for word in
query_words]


corrected_query=' '.join(corrected_words)


print(f"Retrieving information for corrected query:'{corrected_query}' ")


if__name__=="__main__":

user_query="spelingcorrectin algorithm"

dictionary=["spelling","correction","algorithm","information","retrieval","system"]


retrieve_information(user_query, dictionary)
```

Output:-

Retrieving information for corrected query:'spelling correction algorithm'

Practical-4

Evaluation Metrics for IR Systems

- **Calculate precision, recall, and F-measure for a given set of retrieval results.**
- **Use an evaluation toolkit to measure average precision and other evaluation metrics.**

Program:-(4a)

```
def calculate_metrics(retrieved_set,
relevant_set):true_positive=len(retrieved_set.intersection(relevant_set))

false_positive=len(retrieved_set.difference(relevant_set))
false_negative=set.difference(retrieved_set))

'''
```

(Optional) PPTvalues

```
 true_positive=20

false_positive=10

false_negative=30 '''


print("TruePositive:",true_positive,"\nFalsePositive:",false_positive,"\nFalseNeg
ative:",false_negative,"\n")


precision=true_positive/(true_positive+false_positive)
recall=true_positive/(true_positive+false_negative)

f_measure=2*precision*recall/(precision+recall)


returnprecision.recall,f_measure


retrieved_set=set(["doc1","doc2","doc3"])
```

```
relevant_set=set(["doc1","doc4"])

precision,recall,f_measure=calculate_metrics(retrieved_set,relevant_set)

print(f"Precision:{precision}")print(f"Recall:{recall}")
print(f"F-measure:{f_measure}")
```

Output:-

True Positive: 1

False Positive: 2

False Negative: 1

Precision: 0.3333333333333333

Recall:0.5

F-measure: 0.4

Program:-(4b)

```
from sklearn.metrics import average_precision_score


y_true=[0,1,1,0,1,1]
y_scores=[0.1, 0.4, 0.35, 0.8, 0.65, 0.9]
average_precision=average_precision_score(y_true, y_scores)
print(f'Average precision-recall score:{average_precision}')
```

Output:-

Average precision-recall score: 0.8041666666666667

Practical-5

Text Categorization

- **Implement a text classification algorithm (eg. Naive Bayes or Support Vector Machines).**
- **Train the classifier on a labelled dataset and evaluate its performance.**

Program:-(5a)

```
import numpy as np

import re


class NaiveBayesClassifier:

        def_init_(self):

self.vocab=[]

self.positive_counts=None

self.negative_counts=None

self.positive_prior=0

self.negative_prior=0


def preprocess_review(self,review):

        tokens=re.findall(r'\b\w+\b',review.lower())

        return tokens


def vectorize_review(self,review):

        vector=np.zeros(len(self.vocab))

        for word in review:
```

```
            if word in self.vocab:
                vector[self.vocab.index(word)]+=1
            return vector

def train(self,X_train,y_train):
        for review in X_train:
            tokens=self.preprocess_review(review)
self.vocab.extend(tokens)
self.vocab=list(set(self.vocab))

self.positive_counts=np.zeros(len(self.vocab))
self.negative_counts=np.zeros(len(self.vocab))

X_train_vectorized=[self.vectorize_review(self.preprocess_review(review))for
review in X_train]

positive_count=0
    for i in range(len(X_train_vectorized)):
        if y_train[i]=='positive':
self.positive_counts += X_train_vectorized[i]
positive_count += 1
     else:
self.negative_counts += X_train_vectorized[i]
self.positive_prior=positive_count/len(y_train)
self.negative_prior=1-self.positive_prior

    def predict(self, X_test):
```

```
        predictions=[]
        for review in X_test:
review_vectorized=self.vectorize_review(self.preprocess_review(review))
positive_likelihood=np.prod((self.positive_counts/sum(self.positive_counts))**re
view_vectorized)*self.positive_prior
negative_likelihood=np.prod((self.negative_counts/sum(self.negative_counts))**
review_vectorized)*self.negative_prior
            if positive_likelihood>negative_likelihood:
predictions.append('positive')
        else:
predictions.append('negative')
        return predictions
positive_reviews=[
    "The movie was amazing, I like great acting and an engaging plot."
]
Negative_reviews=[
     "Terrible! Would not recommend it to anyone."
]
X_train=positive_reviews + negative_reviews
Y_train=['positive'] * len(positive_reviews) + ['negative'] * len(negative_reviews)

X_test=[
    "The acting was superb!"
]
Y_test=['positive']

classifier= NaiveBayesClassifier()
```

```
classifier.train(X_train,y_train)

predictions=classifier.predict(X_test)

print("Predicted class for the new review:",predictions[0])

accuracy=sum(1 for true_label, predicted_label in zip(y_test, predictions)if
true_label==predicted_label)/len(y_test)

print("Accuracy:",accuracy)
```

Output:-

Predicted class for the new review: positive

Accuracy: 1.0

Practical-6

Clustering for Information Retrieval

- **Implement a clustering algorithm (eg. K-means or hierarchical clustering).**
- **Apply the clustering algorithm to a set of documents and evaluate the clustering results.**

Program:-

```
import numpy as np

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score

Sample documents

documents=[

"Machine learning is the study of coumputer algorithms that improve automatically through experience.",

"Deep learning is a subset of machine learning.",

"Natural language processing is a field of artificial intelligence.",

"Computer vision is a field of study that enables computers to interpret and understand the visual world.",

"Reinforcement learning is a type of machine learning algorithm that teaches an agent how to make decisions in an environment by rewarding desired behaviors.",

"Information retrieval is the process of obtaining information from a collection of documents.",

"Text mining is the process of deriving high-quality information from text.",
```

```
"Data clustering is the task of dividing a set of objects into groups.",
"Hierarchical clustering builds a tree of clusters.",
"K-means clustering is a method of vector quantization."]
vectorizer=TfidfVectorizer()
X=vectorizer.fit_transform(documents)

k=3
kmeans=KMeans(n_clusters=k)
kmeans.fit(X)

silhouette_avg=silhouette_score(X,kmeans.labels_)
print("Silhouette Score:", silhouette_avg)

for i in range(k):
cluster_docs_indices=np.where(kmeans.labels_ ==i)[0]
cluster_docs = [documents[idx] for idx in cluster_docs_indices]
print(f"\nCluster {i+1}:")
for doc in cluster_docs:
print("-",doc)
```

Output:-

Silhouette Score: 0.060681156595772744

Cluster 1:

- Machine learning is the study of coumputer algorithms that improve automatically through experience.
- Deep learning is a subset of machine learning.

- Computer vision is a field of study that enables computers to interpret and understand the visual world.
- Reinforcement learning is a type of machine learning algorithm that teaches an agent how to make decisions in an environment by rewarding desired behaviors.

Cluster 2:

- Natural language processing is a field of artificial intelligence.
- Information retrieval is the process of obtaining information from a collection of documents.
- Text mining is the process of deriving high-quality information from text.

Cluster 3:

- Data clustering is the task of dividing a set of objects into groups.
- Hierarchical clustering builds a tree of clusters.
- K-means clustering is a method of vector quantization.

Practical-7

Web Crawling and Indexing

- **Develop a web crawler to fetch and index web pages.**
- **Handle challenges such as robots.txt, dynamic content, and crawling delays.**

Program:-(7a)

```
import requests

from bs4 import BeautifulSoup


class WebCrawler:

def__init__(self):

self.visited_urls=set()

def crawl(self, url, depth=3):

if depth == 0;

return

if url in self.visited_urls:

return

try:

response=requests.get(url)

if response.status_code ==200:

soup =BeautifulSoup(response.text,'html.parser')

self.index_page(url,soup)
```

```
self.visited_urls.add(url)
for link in soup.find_all('a'):
new_url=link.get('href')
print(f"crawling {new_url}")
if new_url and new_url.startswith('http'):
self.crawl(new_url,depth-1)
except Exception as e:
print(f"Error crawling {url}: {e}")
def index_page(self, url, soup):
title= soup.title.string if soup.title else "No title"
paragraph = soup.find('p').get_text() if soup.find('p') else "No paragraph found"
print(f"Indexing: {url}")
print(f"Title:{title}")
print(f"First Paragraph: {paragraph}")
print("……")
if__name__ == "__main__":
crawler=WebCrawler()
crawler.crawl("https://www.example.com")
```

Output:-

TF-IDF Matrix:

```
[[0.          0.54645401 0.          0.32274454 0.          0.54645401]
[0.54645401 0.          0.32274454 0.          0.54645401 0.        ]
[0.          0.54645401 0.          0.32274454 0.          0.54645401]
[0.          0.          0. 36888498 0.21786941 0.36888498 0.36888498]]
```

Cosine Similarity Matrix:

[[1. 0.10416404 0.07031616]
[0.10416404 1. 0.07031616]
[0.07031616 0.07031616 1.]

Similarity Between Document 0 and Document 1:

0.1041640395072949

Program:-(7b)

```
import requests
from bs4 import BeautifulSoup
import time
from urllib.parse import urlparse

class WebCrawler:
def__init__(self):
self.visited_urls=set()
def crawl(self, url, depth=3, delay=1):
if depth == 0;
return

if url in self.visited_urls:
return

try:
if not self.is_allowed_by_robots(url):
```

```
print(f"Skipping {url} due to robots.txt rules")
return

response=requests.get(url)
if response.status_code == 200:
soup=BeautifulSoup(response.text, 'html.parser')
self.index_page(url,soup)
self.visited_urls.add(url)
for link in soup.find_all('a'):
new_url=link.get('href')
if new_url and new_url.startswith('http'):
time.sleep(delay)
self.crawl(new_url, depth-1, delay)
except Exception as e:
print(f"Error crawling {url}: {e} ")

def is_allowed_by_robots(self,url):
parsed_url=urlparse(url)
robots_url=f"{parsed_url.scheme}://{parsed_url.netloc}/robots.txt"
response=requests.get(robots_url)
if response.status_code==200:
robots_txt=response.text
if "User-agent:*" in robots_txt:
start_index=robots_txt.find("User-agent:*")
end_index=robots_txt.find("User-agent:", start_index+1)
if end_index == -1:
end_index = len(robots_txt)
```

```
relevant_section = robots_txt[start_index:end_index]
if "Disallow:/" in relevant_section:
return False
return True
def index_page(self, url, soup):
title=soup.title.string if soup.title else "No title"
paragraph=soup.find('p').get_text() if soup.find('p') else "No paragraph found"
print(f"Indexing: {url}")
print(f"Title: {title}")
print(f"First Paragraph: {paragraph}")
print("……")

if__name__ == "__main__":
crawler=WebCrawler()
crawler.crawl(" https://www.mercedes-benz.com ")
```

Output:-

Skipping https: //www.mercedes-benz.com due to robots.txt rules

Practical-8

Link Analysis and PageRank

- **Implement the PageRank algorithm to rank web pages based on link analysis.**
- **Apply the PageRank algorithm to a small web graph and analyze the results.**

Program:-

```
def pagerank(graph, damping_factor=0.85, epsilon=1.0e-8, max_iterations=100):
"""
Compute the PageRank of nodes in the graph.
:param graph: A dictionary representing the graph where keys are page IDs and values are lists of outgoing links.
:param damping_factor: The damping factor, typically set between 0 and 1.
:param epsilon: Convergence criterion.
:param max_iterations: Maximum number of iterations.
:return: A dictionary where keys are page IDs and values are their PageRank scores.
"""
```

```
Num_nodes = len(graph)
Pagerank_scores = {node: 1.0 / num_nodes for node in graph}

for_in range(max_iterations):
new_pagerank_scores={}
max_diff=0

for node in graph:
new_pagerank = (1 – damping_factor)/ num_nodes

for referring_page, links in graph.items():
if node in links:
num_outlinks = len(links)
new_pagerank += damping_factor * pageerank_scores[referring_page] /
num_outlinks

new_pagerank_scores[node]= new_pagerank
diff= abs(new_pagerank-pagerank_scores[node])
max_diff=max(max_diff, diff)

pagerank_scores= new_pagerank_scores

if max_diff < epsilon:
break
return pagerank_scores
```

```
web_graph= {
'A': ['B','C '],
'B': ['C '],
'C ': ['A']
}
pagerank_scores = pagerank(web_graph)

sorted_scores = sorted(pagerank_scores.items(), key = lambda x: x[1],
reverse=True)
print("PageRank Scores:")
for node, score in sorted_scores:
print(f"{node}:{score}")
```

Output:-

PageRank Scores:

C: 0.3973996615286747

A: 0.38778970863988693

B: 0.21481062983143845

Practical-9

Learning to Rank

- **Implement a learning to rank algorithm (eg. RanksVM or RankBoost).**
- **Train the ranking model using labelled data and evaluate its effectiveness.**

Program:-(9a)

```
from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


def train_ranksvm(X_train, y_train):

"""

Train RankSVM model using training data.

:paramX_train: Features of training data.

:paramy_train: Labels of training data.
```

```
:return: Trained RankSVM model.
"""
svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
return svm

def predict_ranksvm(model, X_test):
"""
Predict rankings using trained RankSVM model.
:param model: Trained RankSVM model.
:param X_test: Features of test data.
:return: Predicted rankings.
"""
rankings = model.decision_function(X_test)
return rankings

if __name__ == "__main__":
X, y = make_classification(n_samples=100, n_features=20, n_classes=2,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
ranksvm_model = train_ranksvm(X_train_scaled, y_train)
rankings = predict_ranksvm(ranksvm_model, X_test_scaled)
y_pred = (rankings > 0).astype(int)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Output:-

Accuracy: 1.0

Program:-(9b)

```
import numpy as np

from sklearn.svm import SVC

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.metrics import average_precision_score


X, y = make_classification(n_samples=1000, n_features=10, n_classes=2,
random_state=42)


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)


model = SVC(kernel='linear')

model.fit(X_train, y_train)


y_pred = model.predict(X_test)
```

```
map_score = average_precision_score(y_test, y_pred)
print("Mean Average Precision (MAP): ", map_score)
```

Output:-

Mean Average Precision (MAP) : 0.8134754657570191

Practical-10

Advanced Topic in Information Retrieval

- **Implement a text summarization algorithm (eg. Extractive or abstractive).**
- **Build a question-answering system using techniques such as information extraction.**

Program:-(10a)

 # Install Required Libraries

```
bash
pip install nltk gensim
python -m nltk.downloader 'punkt' 'stopwords' 'wordnet'
```

 # Extractive Text Summarization

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from gensim.summarization.keypoints import keywords
```

```
from gensim.summarization import summarize


def extractive_summarization(text, ratio):
# Tokenize text into sentences
sentences = sent_tokenize(text)
# Tokenize text into words
words = word_tokenize(text)
# Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_words =
# Lemmatize words
lemmatizer = WordNetLemmatizer()
lemmatized_words =
# Calculate word frequencies
word_freq = {}
for word in lemmatized_words:
if word in word_freq:
word_freq += 1
else:
word_freq = 1


# Calculate sentence scores
sentence_scores = {}
for sentence in sentences:
for word in word_tokenize(sentence):
if word in word_freq:
if sentence in sentence_scores:
```

```
sentence_scores += word_freq
else:
sentence_scores = word_freq
# Sort sentences by score
sorted_sentences = sorted(sentence_scores.items(), key=lambda x: x,
reverse=True)
# Select top sentences
summary_sentences = ]
# Join summary sentences
summary = ' '.join(summary_sentences)
return summary
# Example usage
text = """
The quick brown fox jumps over the lazy dog. The dog is very happy.
The fox is also happy. They are friends.
"""
ratio = 0.5
summary = extractive_summarization(text, ratio)
print("Summary:", summary)

# Abstractive Text Summarization
from transformers import pipeline
def abstractive_summarization(text, ratio):
summarizer = pipeline("summarization")
result = summarizer(text, ratio=ratio)
return result
# Example usage
```

```
text = """
The quick brown fox jumps over the lazy dog. The dog is very happy.
The fox is also happy. They are friends.
"""
ratio = 0.5
summary = abstractive_summarization(text, ratio)
print("Summary:", summary)
```

Output:-(Extractive)

Summary: The quick brown fox jumps over the lazy dog. The dog is very happy.

Output:-(Abstractive)

Summary: The quick brown fox and the lazy dog are friends and are both happy.

Program:-(10b)

 # Install Required Libraries

```
bash
pip install nltk gensim transformers
python -m nltk.downloader 'punkt' 'stopwords' 'wordnet'
```

 # Code

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from gensim.summarization.keypoints import keywords
from transformers import pipeline
class QuestionAnsweringSystem:
def __init__(self, text):
self.text = text
def extract_entities(self):
# Tokenize text into sentences
sentences = sent_tokenize(self.text)
# Tokenize text into words
words = word_tokenize(self.text)
# Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_words =
# Lemmatize words
 lemmatizer = WordNetLemmatizer()
lemmatized_words =
# Extract entities
entities =
for sentence in sentences:
for word in word_tokenize(sentence):
if word in lemmatized_words:
```

```
entities.append(word)
return entities

def answer_question(self, question):
entities = self.extract_entities()
# Initialize answer
answer = "Unknown"

# Check if question contains entity
for entity in entities:
if entity in question:
answer = entity
break
return answer

# Example usage
text = """
The quick brown fox jumps over the lazy dog. The dog is very happy.
The fox is also happy. They are friends.
"""
question = "Who is happy?"
qas = QuestionAnsweringSystem(text)
answer = qas.answer_question(question)
print("Answer:", answer)
```

Output:-

Answer: dog